

Assignment 10

Aim - To implement the 0-1 Knapsack problem using dynamic programming in C and determine the maximum value that can be obtained without exceeding the given knapsack's capacity.

Analysis

In the 0-1 Knapsack problem, each item has a weight and a value. Each item can either be selected once or not selected. A Dynamic Programming table is used to store the maximum value possible for different numbers of items and capacities.

For every item, two choices are considered

1. Exclude the item
2. Include the item if its weight is within the current capacity.

The maximum of these two choices is stored in the DP table.

Steps

1. Start with n items, their weights and values, and the knapsack capacity.
2. Create a DP table of size $(n+1) \times (\text{capacity}+1)$
3. Initialize the first row and column to 0.
4. For each item and capacity :-
 - If the item's weight is greater than the capacity, exclude it

- Otherwise, calculate both including and excluding the item.
- Store the maximum of the two values.
- 5. The last cell of the table gives the maximum possible value.
- 6. Trace the table backwards and find the selected items.

Time complexity : $O(n \times W)$

Space complexity : $O(n \times W)$

n - No. of items

w - Knapsack capacity

NOTE - DP table requires $(n+1) \times (W+1)$ memory locations.

Code.

```
#include <stdio.h>
```

```
int max (inta, intb) {
```

```
    return (a>b) ? a : b;
```

```
}
```

```
void main() {
```

```
    int n = 8, capacity = 20;
```

```
    int weight[] = {2, 3, 4, 5, 6, 7, 8, 9};
```

```
    int value[] = {3, 4, 5, 8, 9, 10, 11, 13};
```

```
    int dp[9][21];
```

```
    int i, w;
```

```
    for (i = 0; i < n; i++) {
```

```
        for (w = 0; w <= capacity; w++) {
```

```
            if (i == 0 || w == 0) {
```

```
                dp[i][w] = 0;
```

```
            }
```

```
else if (weight[i-1] <= w) {  
    dp[i][w] = max(dp[i-1][w],  
        value[i-1] + dp[i-1][w - weight[i-1]]);  
} else {  
    dp[i][w] = dp[i-1][w];  
}  
  
printf("DP Table : \n\n");  
for(i=0; i<=n; i++) {  
    for(w=0; w<=capacity; w++) {  
        printf("%3d ", dp[i][w]);  
    }  
    printf("\n");  
  
    printf("\n Maximum value = %d\n", dp[n][capacity]);  
    w = capacity;  
    printf("Selected items:");  
    for(i=n; i>0; i--) {  
        if(dp[i][w] != dp[i-1][w]) {  
            printf("%d ", i);  
            w = w - weight[i-1];  
        }  
    }  
    printf("\n");  
}
```

DP Table:

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
0	0	3	4	4	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7	7
0	0	3	4	5	7	8	9	9	12	12	12	12	12	12	12	12	12	12	12	12
0	0	3	4	5	8	8	11	12	13	15	16	17	17	20	20	20	20	20	20	20
0	0	3	4	5	8	9	11	12	13	15	17	17	20	21	22	24	25	26	26	29
0	0	3	4	5	8	9	11	12	13	15	17	18	20	21	22	24	25	27	27	30
0	0	3	4	5	8	9	11	12	13	15	17	18	20	21	22	24	25	27	28	30
0	0	3	4	5	8	9	11	12	13	15	17	18	20	21	22	24	25	27	28	30

Maximum value = 30

Selected items: 6 5 4 1

❖ tarun@Tarun-PC:~/Downloads/ADSA-TSEC-SEM3\$